# MR Whiteboard

Top-level architecture

An infinite whiteboard for the Meta Quest 3 and the Logitech MX Ink stylus, built with Godot 4.7.2. It runs in mixed reality (passthrough), is operated with bare hands, and uses the stylus whenever you draw. The same project also runs as a PC app, so everything except the headset hardware can be tried and tested without a headset. This document describes how the pieces fit together; the README covers using it, `docs/INTERACTIONS.md` every input number, and `docs/SHARING.md` the session protocol. Section 12 proposes improvements.

## 1. The shape of the system

The app is one Godot project, written entirely in GDScript (about 26,000 lines in `src/`, 31 headless test files, GLSL shaders for ink and grid). It has three ideas at its centre:

- **One world, two rigs.** A **Board** node owns the open document and every canvas in the room. It does not know whether a headset or a mouse is attached: a **rig** (XrRig on the Quest, DesktopRig on the PC) feeds it a **PenState** and hand inputs each frame and hosts the interface. This is why the PC build is a faithful test bed.
- **A model that never touches the scene tree. Document**, **FrameData** and **Item** are plain RefCounted data. Everything the user does becomes a **History** operation on the Document, and the Document emits signals that views, autosave, networking and the web page all follow. Undo, sharing and the AI assistant therefore work for free on any edit.
- **Content fixed in the real room.** Each canvas has its own Meta spatial anchor and a pose relative to its room's anchor. The Board finds which room you are in, keeps canvases and panels where you left them across restarts, and re-anchors what you move.

### System context

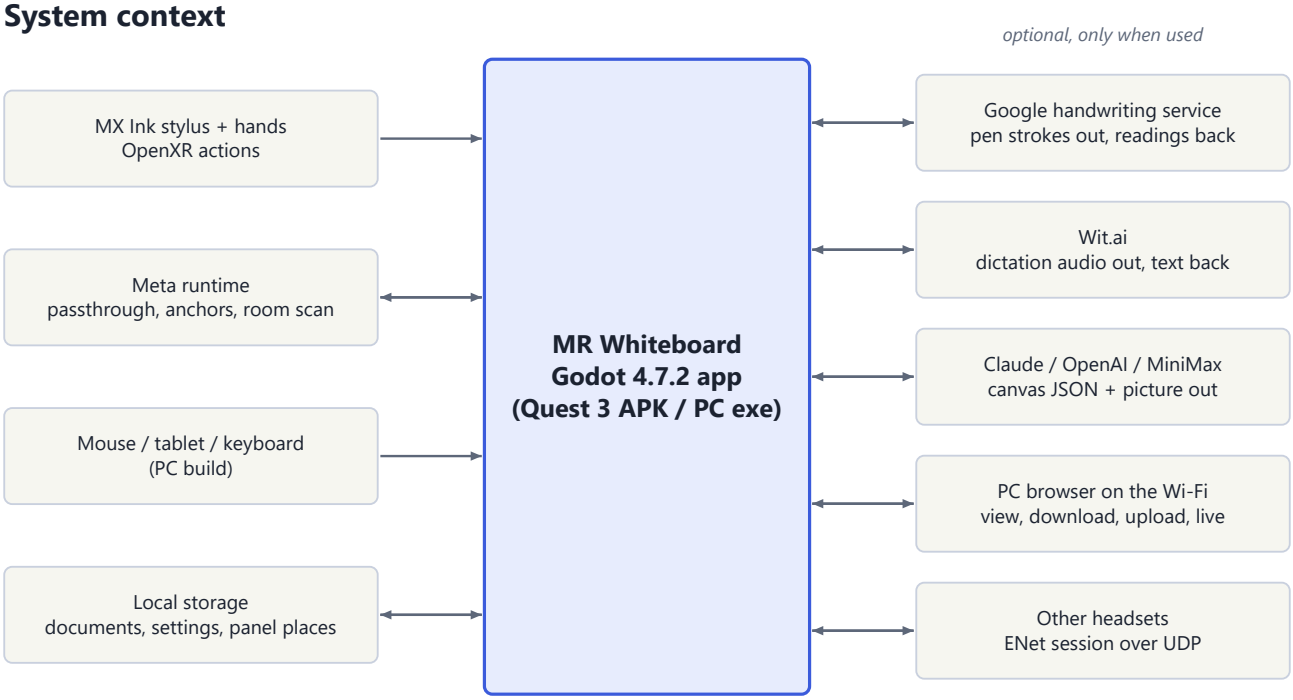



*Figure 1. Everything the app talks to. Only the Meta runtime and the input devices are required; every network service is opt-in and sends only what its feature needs (pen strokes, a dictation's audio, one canvas's description).*

## 2. Layers

Dependencies run downwards only. `core/` depends on nothing above it, which is what makes the headless tests possible; the rigs are the only code that touches the hardware.

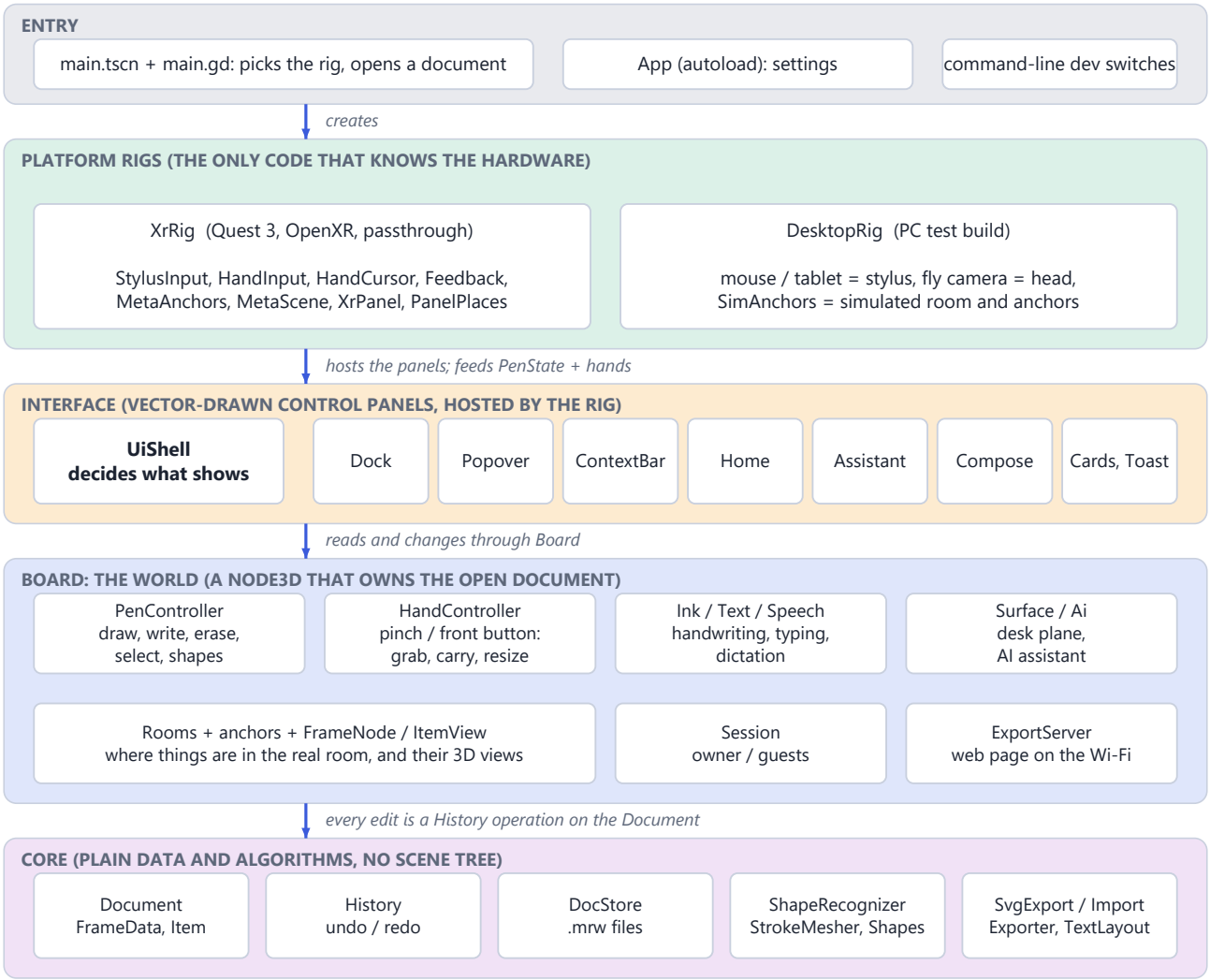



*Figure 2. Layers of the application. The Board is the hub: rigs call it, the interface reads and drives it, and it owns the document and the controllers.*

## 3. Modules

Source layout under `src/`, with the classes that carry each responsibility.

| Folder | Responsibility | Key classes |
|---|---|---|
| core/ | Document model, undo, files, geometry, shape recognition, ink meshing, SVG in and out, the web server and its pages. | Document, FrameData, Item, History, DocStore, ShapeRecognizer, StrokeMesher, Shapes, TextLayout, SvgExport, SvgImport, ExportServer, WebPages, InputTuning |
| board/ | The world: rooms, anchors, frame views, selection, and everything that edits the document from input. | Board, PenController, HandController, InkController, TextController, SpeechController, SurfaceController, AiController, FrameNode, ItemView, PeerAvatars, LivePreview, AnchorService, SimAnchors |
| xr/ | Quest 3 only: OpenXR start-up, stylus and hands, spatial anchors, room scan, floating panels. | XrRig, StylusInput, StylusModel, StylusRay, HandInput, HandCursor, Feedback, MetaAnchors, MetaScene, XrPanel, PanelPlaces |
| desktop/ | PC only: mouse as stylus, fly camera as head, simulated room. | DesktopRig |

| Folder | Responsibility | Key classes |
|---|---|---|
| ui/ | The interface as vector-drawn Control trees; no bitmaps. UiShell decides which panel shows when. | UiShell, FloatPanel, Dock, Popover, ContextBar, InkChip, SuggestionBar, HomePanel, AssistantPanel, ComposePanel, SessionPage, GuidePage, Btn, Icons, UITheme, VKeyboard |
| net/ | Shared sessions: roles, wire, replication, shared coordinate frame, discovery, live presence. | Session, SessionNet, DocSync, SharedSpace, LanDiscovery, LiveOut |
| ai/ | Language-model assistant: request builders, HTTP client, canvas description, answer to items. | LlmApi, LlmClient, CanvasBrief, AiPrompts, AiReply |
| ink/, speech/ | Pluggable recognisers behind small interfaces; writing-versus-drawing detection; microphone capture. | InkRecognizer, GoogleInk, WritingDetector, SpeechRecognizer, WitSpeech, MicRecorder |
| shaders/ | Anti-aliased ink ribbons and tubes, neon, the fading grid. | ink_canvas, ink_tube, grid |

# 4. Start-up and the frame loop

## Start-up

- **App** is the only autoload. It holds settings (`user://settings.cfg`, a table of defaults) and emits `setting_changed`.
- `main.gd` asks whether OpenXR is running. It creates the **Board**, then either an **XrRig** (with MetaAnchors) or a **DesktopRig** (with SimAnchors), and hands the Board an **AnchorService**, an abstract class with those two implementations.
- It opens the last document (or the demo, or a new one), shows the guide on the very first start, and resumes a saved session if there was one.
- Each rig builds a **UiShell** and hosts its panels: on the PC as overlays on the window, in the headset as XrPanel quads or compositor layers floating in the room.

## Every frame

The rig reads its devices (stylus, hand joints, or mouse), smooths and normalises them, and calls `Board.tick(pen, hands, dt)`. The Board then runs, in order:

- **Rooms and anchors:** decide which room you are in, place every canvas from its anchor, reconcile anchors that are lost or stale.
- **Session:** send and receive presence and edits.
- **Exactly one input mode** owns the pen: placing a canvas, finding the desk, aligning a session, or normal work, where **HandController** (grabs, shared by hands and the stylus front button) runs before **PenController** (draw, write, erase, select), so a grab blocks a stroke.
- **Hover and ink:** update the proximity fade of grids and borders; handwriting conversion timers run.
- **Autosave:** a document is saved 1.5 s after the last change, immediately when the headset comes off.

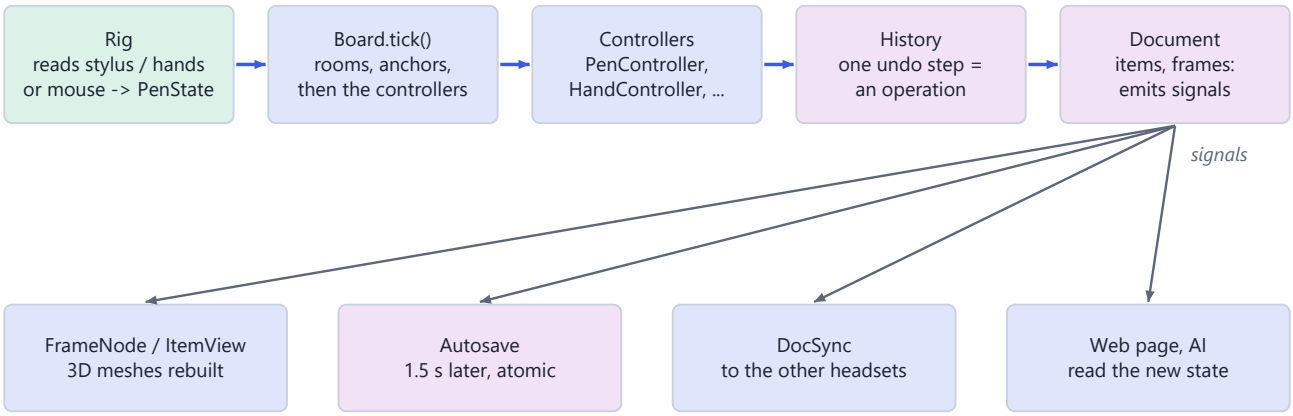



*Figure 3. The path of one edit. Whatever starts an edit, it reaches the Document through History, and the Document's signals are the only thing the rest of the app follows.*

## 5. The document model and storage

- **Document** holds *rooms*, *frames* and *items*. A **room** is a place in the real world (its anchor, the name). A **frame** (**FrameData**) is a canvas, a coordinate system with a pose in its room, a scale, a size that grows with content, an optional desk surface, and its own anchor id. An **Item** is a stroke, a parametric shape (box, line, arrow, polygon...) or a text, with style, geometry and z order held in plain fields so it can be copied and diffed.
- **History** keeps 200 levels of undo and redo as operations (add, remove, modify, transform, frame changes, groups). Handwriting conversion, an accepted AI suggestion, an SVG import and a web-page delete are each one step.
- **DocStore** reads and writes `.mrw` files in `user://docs`: JSON, with stroke points packed as base64 float arrays. A save cut short is recovered from its `.tmp` / `.bak` copy. Spatial anchors live in the headset's own storage; documents keep only their UUIDs, shared between duplicates and erased only when no document uses them.
- The AI assistant's suggestion layers and conversation sit in a separate file beside the document (`<id>.ai.json`), never in the document until the user accepts a suggestion.
- Also on disk: `settings.cfg` and `panel_places.json` (where the dock and side panels were left).

## 6. Space: rooms, anchors and panels

This is the most delicate part of the system. Its rules, all in `Board` and the `AnchorService` implementations:

- **Frames are placed from anchors.** A frame's world pose is its room's pose times its own pose in the room; while the room is tracked, each canvas is corrected from its own anchor (over 1 cm or 0.5 degrees), so a canvas follows the real room.
- **Room recognition.** A room is where its own anchor is; failing that, where its canvases' anchors agree. A Quest 3 maps the whole home, so several rooms are found at once: the one you stand nearest to wins. A short dropout keeps everything in place; an anchor still being placed after the headset is put on is waited for (up to 30 s); only a room not found within 10 s is shown in front of you, turned about the vertical only.
- **Moving re-anchors.** Letting go of a moved canvas creates a new anchor and erases the old one once the new one is tracked. Failed creations are retried after 3, 6, 12 ... 60 s.
- **Panels** (dock, Home, assistant) have an anchor of their own, kept by **PanelPlaces**, so they also survive a restart or recentre.
- **Desk canvases** are bound to a plane fitted through points the stylus nib felt (**PlaneFit**, **SurfaceController**); every grab is projected onto that plane, and each nib touch while drawing refines it.
- **MetaAnchors** tracks stored anchors itself rather than through the plugin's manager, whose re-tracking fails after the headset has been off.

# 7. Input and interaction

One vocabulary serves every input device: **point**, **act** (draw or press), **grab** (a pinch, or the stylus front button), **undo** (stylus back button) and **menu** (palm up). Devices are reduced to the same two types before the Board sees them:

- **PenState**: tracked, source (stylus, mouse or pinch), nib position and direction, pressure, down, contact, front and back buttons. **StylusInput** fills it from the Logitech MX Ink OpenXR actions with hysteresis; DesktopRig fills it from the mouse; a pinch can stand in when no stylus is present.
- **HandInput**: pointing ray, pinch strength and stabilised pinch point, fingertip, palm direction, one per hand, from the OpenXR hand-interaction profile with the joint tracker as fallback.
- A pointer **touches** a canvas within 12 cm (and grabs 1:1) and otherwise **points** (what it grabs moves further than the hand, by the ratio of distances from the head). Every threshold is a constant in **InputTuning**, so a tuning pass is a one-file change.
- Text arrives from three producers that all end as Items: the on-screen or Bluetooth keyboard (**TextController**), dictation (**SpeechController** with **MicRecorder** and **WitSpeech**) and handwriting (**InkController** with **WritingDetector** and **GoogleInk**).

# 8. Interface

The work is the canvas, so the interface stays small and floats where it is needed. **UiShell** owns no layout; it decides which **FloatPanel** is visible and what it shows, and the rig positions each one. FloatPanel kinds are dock, popover, side, canvas (pinned 12 mm in front of a point on a canvas), toast and compose. In the headset each panel renders into its own SubViewport at 2x and is presented on an OpenXR compositor layer (a textured quad as fallback); pokes and rays become ordinary mouse events, so every Control works unchanged. Panels fade back while the pen draws or a hand carries something. All controls are vector-drawn (`Btn`, `Icons`, `UITheme`) and text uses one SDF font via `TextLayout.font()`, so nothing is a bitmap and everything stays sharp at any distance.

# 9. Services and sessions

## External capabilities

| Capability | Implementation | Extension point |
|---|---|---|
| Handwriting to text | GoogleInk posts pen strokes with timing to Google's handwriting input endpoint; several readings come back. | InkRecognizer |
| Speech to text | MicRecorder cuts speech at silence, WitSpeech sends 16 kHz PCM to Wit.ai. | SpeechRecognizer |
| AI assistant | CanvasBrief describes the canvas as JSON plus a picture; LlmApi builds Claude / OpenAI / MiniMax requests; AiReply turns answers into items; AiController keeps suggestion layers. | one more provider entry in LlmApi |
| Web page | ExportServer is a TCP/HTTP server inside the app (port 8080, access code, same-site cookie, CSP). Serves SVG, zip, live view, upload, AI page. | WebPages / WebAssets |
| Export / import | SVG per canvas, glTF .glb, .mrw; SVG import creates real shapes and strokes as one undo step. | Exporter, SvgImport |

## Shared sessions

An owner and any number of guests on one Wi-Fi edit the same canvases. The owner's document is the truth. Replication sends *state, not operations*: **DocSync** listens to the Document's signals, remembers which ids changed, and sends their current state; the receiver applies it without echo. A batch that arrives twice or late does no harm, and undo, imports and accepted AI suggestions replicate with no extra code. Guests edit a *transient* copy of the shared room that is never saved into their documents; the host validates each edit (may this guest draw? is that canvas locked?) before applying and forwarding it.

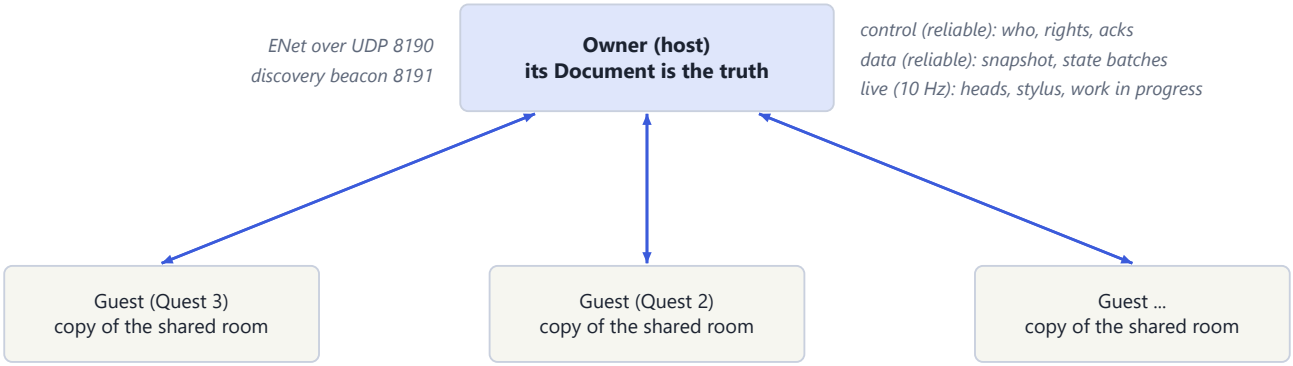



*Figure 4. Session topology. SessionNet uses Godot's ENet as a packet pipe with three channels, so a large snapshot never delays a pointer.*

- **SharedSpace** holds the maths of agreeing on a common frame: everyone touches the same two real places, so each headset can convert the wire's shared coordinates to its own tracking space.
- **LiveOut** and **LivePreview** stream and show work in progress (a stroke growing, a canvas being carried) until the real edit arrives and takes over.
- Everything received is somebody else's data: dictionaries only (no objects), bounded, validated. Nothing is encrypted, so it is meant for a trusted network, with a four-digit code and rate-limited retries.

## 10. Quality, build and verification

- **Tests.** `pwsh tests/run_all.ps1` runs each `tests/test_*.gd` as a Node under `tests/runner.tscn` (autoloads do not exist for plain scripts). They cover the model, recogniser, export, the web server over a real local connection, a simulated pen driving the whole Board, the UI, anchoring and room choice, the desk, and a three-board session over real local ENet connections. The AI is tested against sample answers, without the network.
- **Checks.** `godot --headless --path . -s tools/dev/check_scripts.gd` compiles every script. `--screenshot`, `sim_draw` and `sim_session` render and drive the PC app for checking by eye.
- **Build.** `tools/quest.ps1` exports and installs the APK; `tools/pc.ps1 -Export` builds the exe. Nothing under `src/` may depend on `tools/` or `tests/`, which are left out of exports. The Godot OpenXR Vendors plugin (Meta and Windows binaries) is used only where Godot lacks a core equivalent: passthrough, Meta anchors, room scan.
- **Verification status.** Everything runs and is tested on the PC. The headset-specific code (`src/xr/`, anchors after the headset has been off, stylus geometry, the desk force threshold, UDP sessions between real headsets, dictation, and real requests to the AI providers) is written from the APIs and was checked only partly on a device. README, section "Status", lists what to check first.

## 11. Conventions that shape the code

- A new tunable number for hands, stylus or menu goes in **InputTuning**; behaviour is documented in `docs/INTERACTIONS.md`.
- Recognisers sit behind small interfaces (**InkRecognizer**, **SpeechRecognizer**, **AnchorService**, provider entries in **LlmApi**) so that online services, the PC simulation and future on-device models are interchangeable.
- Every mutation of a Document goes through **History** (or is replicated state), never straight to the views.
- The in-headset guide (**GuidePage**) is data: a feature that changes updates its topic, and a smoke test opens every topic.
- GDScript traps seen here: lambdas capture locals by value; `:=` needs a typed right-hand side; `convert` and `duplicate` clash with built-ins.

# 12. How the architecture can be improved

The foundations are sound: the model is free of the scene tree, every edit goes through History, the hardware sits behind two rigs and small interfaces, and replication sends state. The proposals below keep those ideas and deal with what has grown around them. Each one is based on the current code; sizes are line counts of `src/`. None of them is a known bug; they reduce the cost and risk of the next round of features and of first contact with the headset.

### At a glance, in the suggested order

| | Change | Effort | Why now |
|---|---|---|---|
| 1 | Document version check and migration ladder (12.2) | Small | Protects users' files before the first format change |
| 2 | Measure frame time; move saving, snapshots and web work off the main thread (12.3) | Medium | Frame drops are felt in VR; the first headset runs will show it |
| 3 | Recorder and player for headset input (12.6) | Medium | Turns the coming device testing into lasting tests |
| 4 | Extract RoomTracker, then the rest of Board (12.1) | Medium-large | The riskiest logic, about to be debugged on the device |
| 5 | Typed settings; web server and settings out of core (12.4, 12.5) | Small-medium | Mechanical, low risk; tidy before more features |
| 6 | Split Session into role objects (12.1) | Medium | Before sessions gain features |
| 7 | Per-item revisions, then DTLS and TLS (12.7) | Medium-large | When sessions are used beyond one trusted network |
| 8 | Finer signals, DevLaunch (12.8) | Small | Whenever convenient |

## 12.1 Split the two largest classes

**Board** (1,851 lines, 103 functions) and **Session** (1,727 lines, 109 functions) each hold several responsibilities, already separated by comment banners. Board's *rooms & anchors* section alone is about 600 lines, and it is the most delicate and least device-verified logic in the app. Moving each banner into its own class, with the original class kept as a thin coordinator, makes each part testable on its own (RoomTracker against SimAnchors with no Board) and shrinks what has to be read to change one behaviour. Session's `role` checks become two role objects, HostRole and GuestRole, so host-only and guest-only state can no longer be confused.

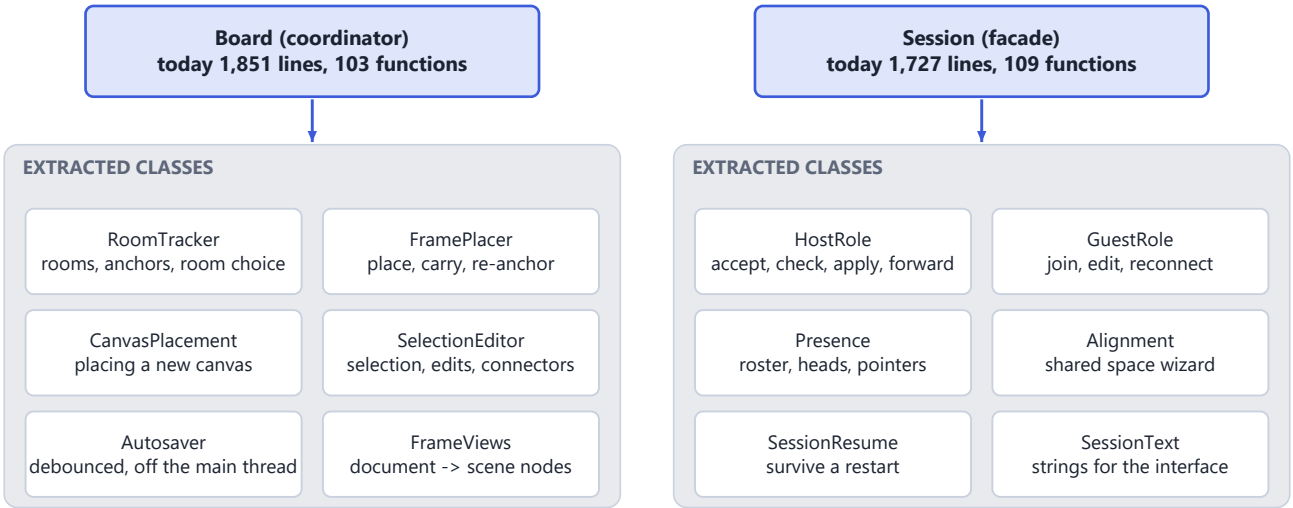



*Figure 5. A possible split. The public surface of Board and Session stays the same, so the interface, rigs and tests change little; it can be done one extraction at a time, with the existing tests as the safety net.*

- Two controllers call a private Board function (`board._release_glue_to` from PenController and AiController). Connector gluing deserves a small public service of its own; it is the first candidate to move out.

### 12.2 Give the document format a migration path

`Document.to_dict()` writes `"version": 1`, but loading checks only `format` and never reads the version. The first incompatible change will then either break old `.mrw` files or need guesswork. Add a migration ladder now, while it is cheap: `from_dict` reads the version, applies `migrate_1_to_2`, `migrate_2_to_3` ... in order, and refuses a newer version with a clear message rather than dropping fields. Keep one sample file per version under `tests/`. The same applies to the `.ai.json` file and to the session snapshot, which carries a whole document between app versions.

### 12.3 Keep heavy work off the frame

No code in `src/` uses threads. On the Quest at 90 Hz a frame has about 11 ms, and several jobs grow with the document and run inside one frame: the autosave (whole document to JSON, then written), the session snapshot (JSON then gzip, up to 256 MB unpacked), and the web server building SVG files, thumbnails and zips in `_process`. Measure first with a large document on the headset; then take a cheap snapshot (the dictionary) on the main thread and do the serialising, compressing and writing on `WorkerThreadPool`. The atomic `.tmp` / `.bak` save already makes a background write safe.

### 12.4 Make the layers true to their names

- **Move the web server out of core.** ExportServer, WebPages and WebAssets (about 2,100 lines) are an HTTP server and a web front end. ExportServer is the only Node in `core/`. A `src/web/` folder states what they are and keeps `core/` to data and algorithms. Their callables into the Board already form a clean seam.
- **Keep settings out of core.** PenState reads the `App` autoload to decide on haptics, the only core class that does. The rig can pass that flag in, so core runs with no autoloads at all.

### 12.5 Type the settings

44 different settings are read as `App.val("key")` in 25 files. A misspelt key returns `null` without complaint, the type is lost at each call site, and a hand-edited or old `settings.cfg` can put a value of the wrong type into the app. A typed facade (`Settings.brush_width: float`, or at least constants for the keys) and a type check against DEFAULTS when loading would catch both, with no change to the file format.

### 12.6 Turn headset sessions into tests

All the XR code is verified only on the PC. The rigs already reduce the hardware to **PenState** and **HandInput** each frame, which is an ideal recording point. A small recorder that writes those streams to a file, plus the anchor events, during a headset session, and a player that feeds them to a Board in `tests/runner.tscn`, would turn every bug seen on the device into a repeatable regression test (anchors slow to locate after donning, the wrong room chosen). Keep the recorder in `src/` behind a setting, since `tools/` is not exported.

### 12.7 Sessions: concurrent edits and transport security

- **Concurrent edits.** DocSync replaces whole items, and the host applies batches in arrival order, so when two people change the same item at once (one recolours a box while the other moves it), the later batch silently undoes the other person's change. A per-item revision number lets the host detect this and either merge by field (colour vs. position) or tell the loser. Field-level merging fits the state-based design without becoming a full CRDT.
- **Encryption.** Sessions use plain ENet over UDP and the web page uses plain HTTP; the four-digit code keeps casual visitors out but does not protect the content. Godot's ENet supports DTLS (`ENetConnection.dtls_server_setup`) and `StreamPeerTLS` can wrap the web server's connections. A self-signed certificate made on first start, with its fingerprint checked through the session code, would protect sessions on shared networks such as schools.

### 12.8 Smaller items

- **Finer change signals.** `Board.changed` marks every visible panel for refresh. Cheap today; as panels grow, separate signals (selection, tool, room, session) would refresh only what depends on them.
- **Separate development switches from start-up.** `main.gd` mixes product start-up with about 20 development switches; a `DevLaunch` helper would keep the real start-up path short.